SPRINT 12 · SERIES, LENGUAJE Y GENERATIVA · S12.2

NLP clásico y embeddings

TurisData recibe reseñas de huéspedes y nadie puede leerlas todas. ¿De qué se quejan? ¿Cuál es el sentimiento?

Los datos: reseñas sintéticas

`resenas.csv`: texto, **nota** (1–5), **sentimiento** (positivo / neutro / negativo) y **temas** mencionados.

> Son reseñas **sintéticas** (inventadas para el curso): más repetitivas y fáciles que las reales. Por eso una exactitud muy alta no hay que creérsela del todo.

Limpieza y tokenización

El ordenador ve «Bueno», «bueno!» y «BUENO» como palabras distintas.

```
def limpiar(texto):
    texto = texto.lower()
    texto = re.sub(r"[^a-záéíóúüñ\s]", " ", texto)
    return " ".join(texto.split())

top5 = Counter(tokens_todos).most_common(5)          # Ejercicio 2
```

- **Tokens**: las palabras. **Stopwords** («de», «el»…): casi no aportan, **pero no quites «no»**.

De texto a números: BoW y TF-IDF

Bolsa de palabras

Una columna por palabra; cada reseña cuenta cuántas veces aparece. Pierde el orden, pero es muy útil.

TF-IDF

Igual, pero las palabras que salen en **todas** las reseñas pesan menos y las poco frecuentes pesan más.

El vectorizador se ajusta **solo con el entrenamiento** (misma idea que el escalado sin fuga).

Clasificar el sentimiento

```
modelo_lr = make_pipeline(TfidfVectorizer(preprocessor=limpiar),
                          LogisticRegression(max_iter=1000))     # Ejercicio 3
modelo_nb = make_pipeline(CountVectorizer(preprocessor=limpiar),
                          MultinomialNB())
acc_lr, acc_nb = modelo_lr.score(X_pru, y_pru), modelo_nb.score(X_pru, y_pru)
```

Ambos son **explicables**: los pesos de la regresión dicen qué palabras empujan hacia «negativo».

Los límites: negaciones y n-gramas

Una bolsa de palabras **no entiende el orden**: «no es impecable» se parece a «es impecable».

- Ejercicio 4: ¿trata el modelo igual las frases con y sin negación? (`falla_negacion`).
- **Bigramas** (`ngram_range=(1, 2)`): «no fue» mantiene la negación junto a la palabra.
- Los bigramas y trigramas más frecuentes en las reseñas **negativas** revelan quejas concretas («no compensa el precio»).

Temas por palabras clave

- Sin entrenar nada: para cada tema, una lista de palabras que lo delatan.
- **Transparente y barata**; su límite es que hay que **mantener la lista**.
- Ejercicio 5: `tema_peor` (menor F1) y `n_precio` (reseñas que mencionan realmente el precio). Mira los fallos y decide qué cambiar en el diccionario.
- Mini-reto: panel de `quejas` = nº de reseñas **negativas** por tema, de mayor a menor, y `queja_principal`.

Embeddings: de la bolsa de palabras al vector denso (S12_04)

- TF-IDF da a cada reseña un vector **enorme y casi vacío**; «ruidoso» y «no nos dejó dormir» quedan *lejos*.
- Un **embedding** representa el texto con **pocos números** y deja cerca lo parecido.

```
lsa = TruncatedSVD(n_components=12, random_state=SEMILLA)       # Ejercicio 1
Z = lsa.fit_transform(M)                                        # (1500, 12)
def parecidas(i, k=3): ...    # coseno sobre Z, sin contar la propia reseña (Ejercicio 2)
```

Similitud del coseno: 1 = misma dirección, 0 = sin relación; no depende de la longitud. Es **ampliación para casa**.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S12_03_nlp_clasico — 55 min

Reseñas: limpieza, TF-IDF, sentimiento, temas y n-gramas

sprints/S12/notebooks/S12_03_nlp_clasico.ipynb